



Introduction to Scientific Computation

Lecture 1

Fall 2026

Complexity, Floating-point arithmetic
Numerical stability, Linear Systems

Complexity

One of definitions: it is a measure of computational cost
In order to give a more formal definition quite a big-O formalism should be introduced (see recommended materials for this week).
We will operate with a concept.

Complexity

The notion of complexity is about how good or bad is the algorithm.
We want to think in an abstract way, independent of:

- implementation details (Python, C++, matlab)
- hardware (stm32, intel Pentium D, Intel Core i7-8700K)
- etc and whatnot

Complexity

Let us use constant time operations (elementary operations).

E.g. + and *

Remember: real runtime doesn't matter (does it ?)

$$3 + 5 = \begin{array}{r} 0011 \\ + 0101 \\ \hline 1000 \end{array}$$

Complexity

Let us use constant time operation (elementary operations).

E.g. + and *

Remember: real runtime doesn't matter (does it ?)

$$3 + 5 = \begin{array}{r} 0011 \\ + 0101 \\ \hline 1000 \end{array}$$

$$T(n) = c \cdot n$$

c - is a cost for summation of one bit

Complexity

Let us use constant time operation (elementary operations).

E.g. + and *

Remember: real runtime doesn't matter (does it ?)

$$3 + 5 = \begin{array}{r} 0011 \\ + 0101 \\ \hline 1000 \end{array}$$

$$T(n) = c \cdot n$$

c - is a cost for summation of one bit

c_1 - for cpu1

c_2 - for cpu2

Complexity

In order to get rid of c we will use **Asymptotic complexity** with big-O notation.

Complexity

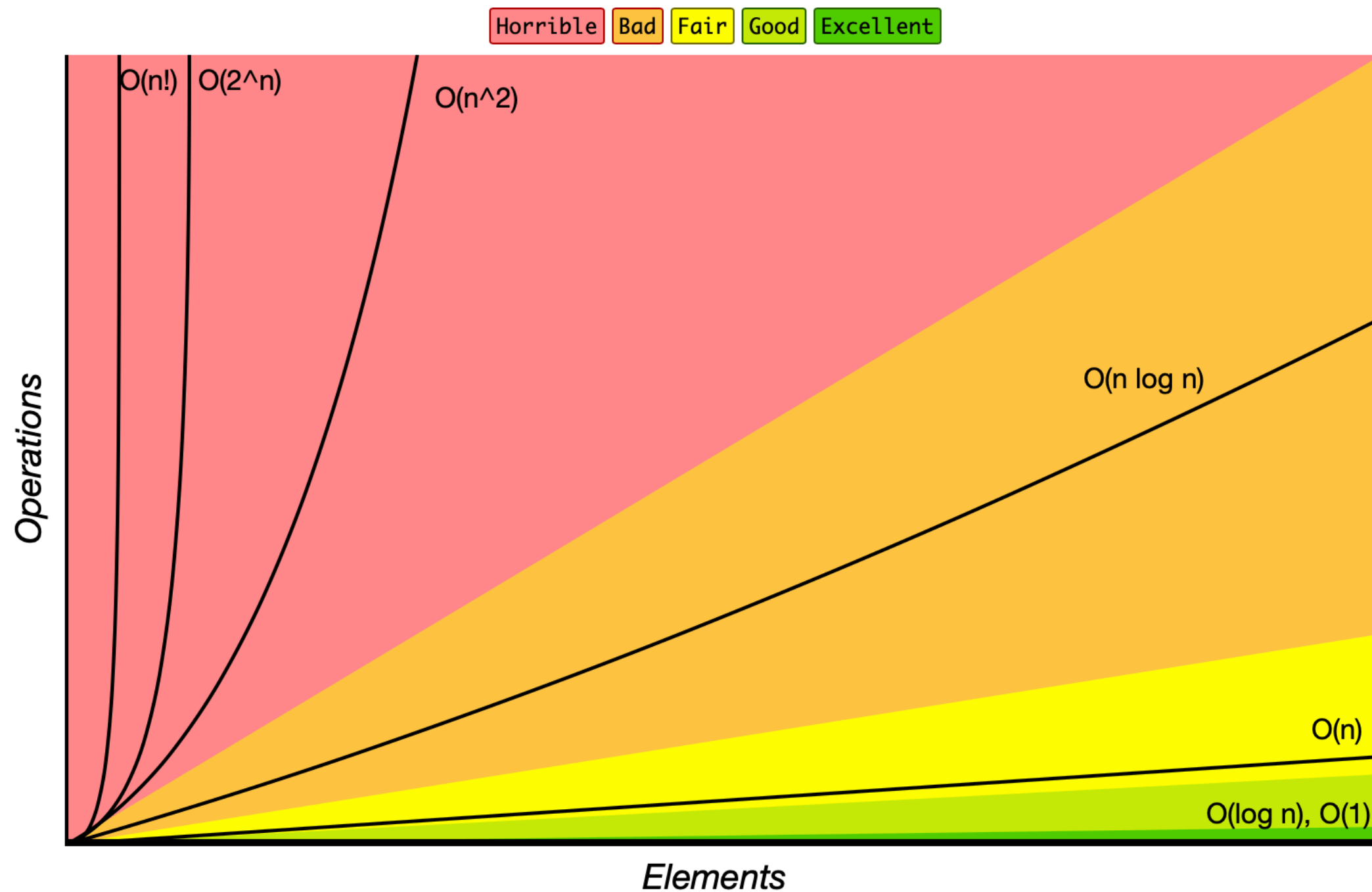
In order to get rid of c we will use **Asymptotic complexity** with big-O notation.

Definition: for any monotonic functions $f(n)$ and $g(n)$ from positive integers to the positive integers $f(n) = O(g(n))$ when there exist constants $c > 0$ and $n_0 > 0$ such that

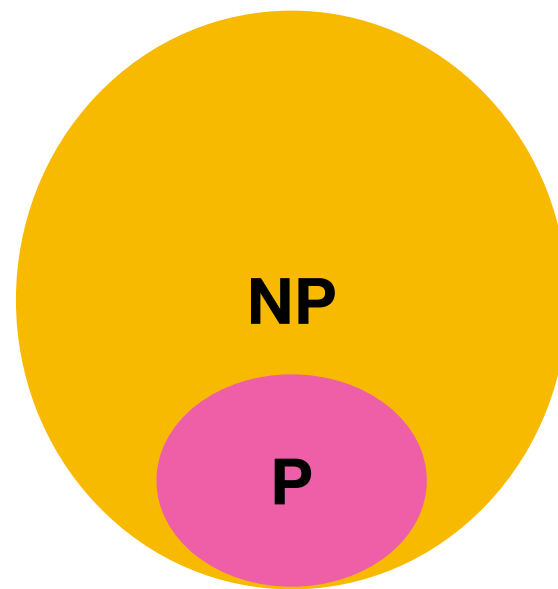
$$f(n) \leq c \cdot g(n), \text{ for all } n \geq n_0$$

Complexity

Big-O Complexity Chart



Complexity



???

Representation of numbers

Fixed point

The most straightforward format for the representation of real numbers is **fixed point** representation, a.k.a **Qm.n** format.

Qm.n number is in the range $[-2^m, 2^m + 2^{-n}]$ with the resolution 2^{-n} requires $m + n + 1$ bits for storage.

Representation of numbers

Floating point

We are mostly interested in **floating point numbers** represented as e.g.

$$1.2345 = \underbrace{12345}_{\text{significand}} \times \underbrace{10}_{\text{base}} \overbrace{-4}^{\text{exponent}}$$

IEEE 754

Single and double precision

$$X = \pm 1.\overline{b_1 b_2 \dots b_K} \cdot 2^e$$

$$X \in [X - \Delta X, X + \Delta X],$$

Single

- S EEEEEEEEE FFFFFFFFFFFFFFFFFFFFFFFF
- 0 1 8 9 31

Absolute accuracy

$$\Delta X \leq \frac{1}{2} 2^{-K} * 2^e \leq |X| \cdot 2^{-K-1}$$

Double

- S EEEEEEEEEEE FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
- 0 1 11 12 63

Relative accuracy

The **relative accuracy** of single precision is $10^{-7} - 10^{-8}$, $\frac{\Delta X}{X} \leq 2^{-K-1}$,
while for double precision it is $10^{-14} - 10^{-16}$.

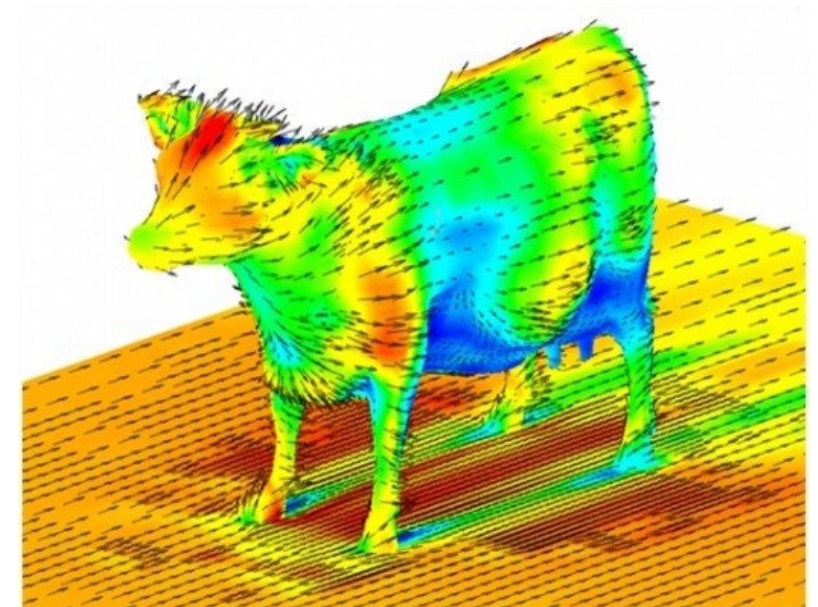
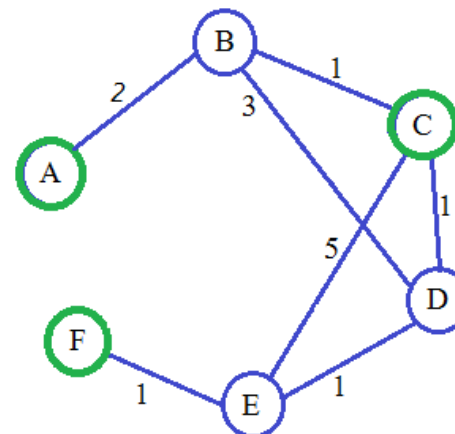
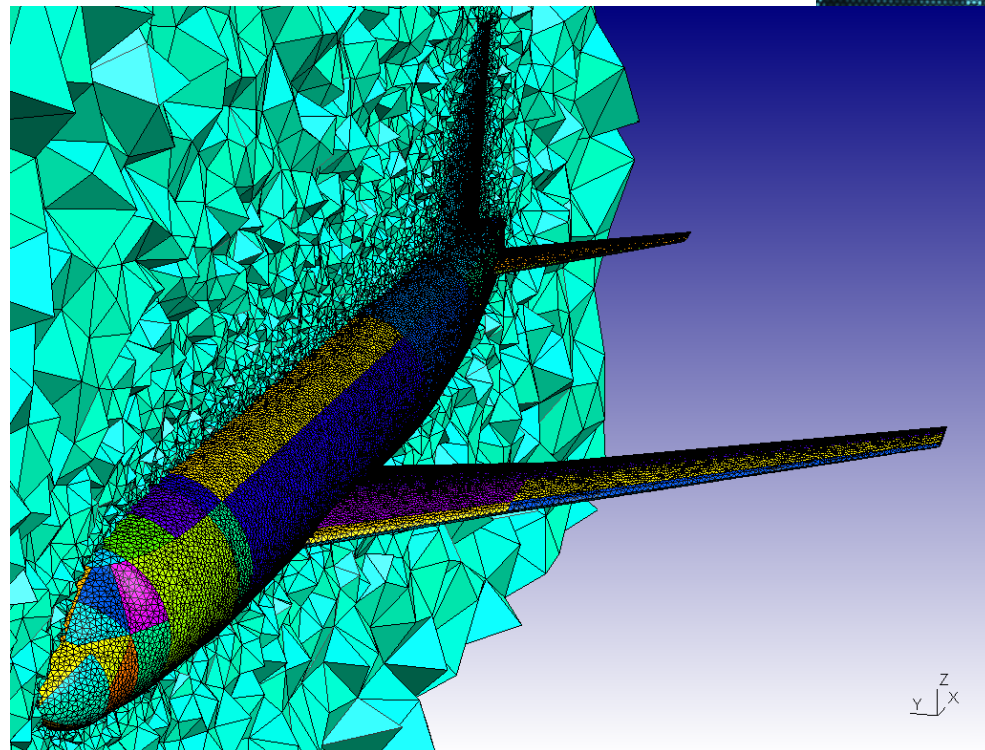
A **float32** takes **4 bytes**; **float64**, or double precision, takes **8 bytes**.

These are the two common floating point-types supported in hardware (float4, 8 and 16 are getting traction on GPUs TPUs and NPUs).

You should use **double precision** in CSE and **single** in GPU programming / Data Science.

Format	Total bits	Significand bits	Exponent bits	Smallest number	Largest number
Single precision	32	23 + 1 sign	8	ca. $1.2 \cdot 10^{-38}$	ca. $3.4 \cdot 10^{38}$
Double precision	64	52 + 1 sign	11	ca. $5.0 \cdot 10^{-324}$	ca. $1.8 \cdot 10^{308}$

Systems of Linear Equations



Pop Quiz: Linear Algebra Intuition

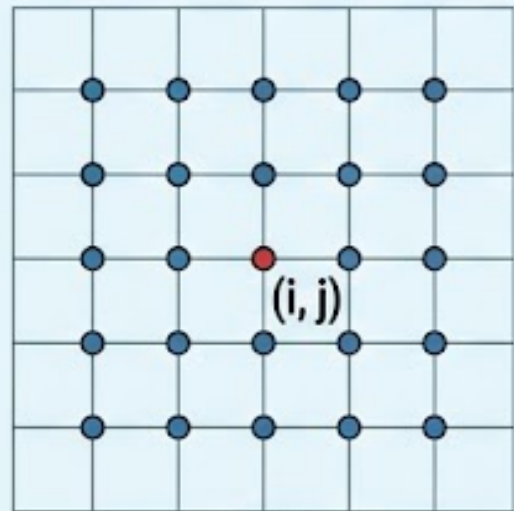
- 1. What is linearity (as in “linear algebra”)?**
- 2. What is a matrix?**
- 3. Why is matrix multiplication defined the way it is?**

Some ways linear systems arise in scientific computing

1. Discretization of differential equations

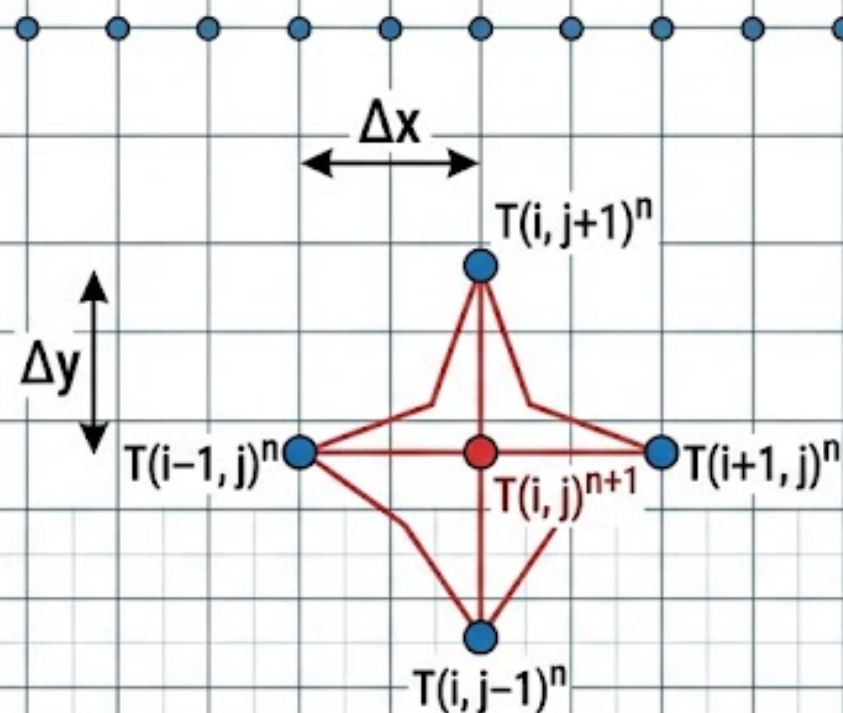
NUMERICAL SCHEME FOR 2D HEAT EQUATION: FINITE DIFFERENCE METHOD (EXPLICIT)

SPATIAL DISCRETIZATION



$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

$$\frac{\partial T}{\partial t} \approx \alpha \left[\frac{T(i+1, j) - 2T(i, j) + T(i-1, j)}{\Delta x^2} + \frac{T(i, j+1) - 2T(i, j) + T(i, j-1)}{\Delta y^2} \right]$$



$$T(i, j)^{n+1} = T(i, j)^n + \alpha \Delta t \left[\frac{T(i+1, j)^n - 2T(i, j)^n + T(i-1, j)^n}{\Delta x^2} + \frac{T(i, j+1)^n - 2T(i, j)^n + T(i, j-1)^n}{\Delta y^2} \right]$$

STABILITY CONDITION:

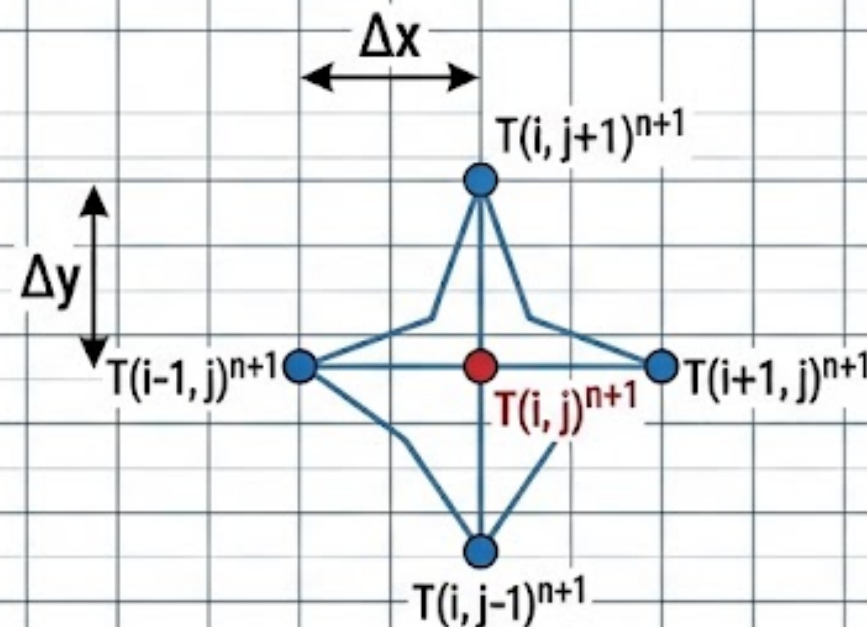
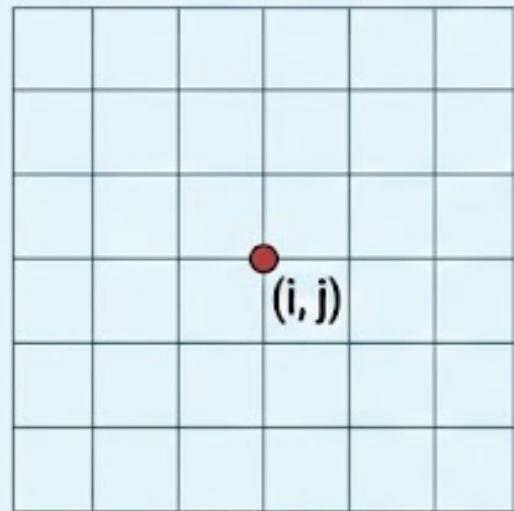
$$\Delta t \leq \min \left(\frac{\Delta x^2, \Delta y^2}{4\alpha} \right)$$

Some ways linear systems arise in scientific computing

1. Discretization of differential equations

NUMERICAL SCHEME FOR 2D HEAT EQUATION: FINITE DIFFERENCE METHOD (IMPLICIT)

SPATIAL DISCRETIZATION



$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

$$\frac{T(i, j)^{n+1} - T(i, j)^n}{\Delta t} \approx \alpha \left\{ \frac{T(i+1, j)^{n+1} - 2T(i, j)^{n+1} + T(i-1, j)^{n+1}}{\Delta x^2} + \frac{T(i, j+1)^{n+1} - 2T(i, j)^{n+1} + T(i, j-1)^{n+1}}{\Delta y^2} \right\}$$

Forming a system of equations ($Ax = b$):

$$-\lambda_x T(i-1, j)^{n+1} - \lambda_y T(i, j-1)^{n+1} + (1 + 2\lambda_x + 2\lambda_y) T(i, j)^{n+1} - \lambda_x T(i+1, j)^{n+1} - \lambda_y T(i, j+1)^{n+1} = T(i, j)^n$$

where $\lambda_x = \alpha \Delta t / \Delta x^2$ and $\lambda_y = \alpha \Delta t / \Delta y^2$

$$A = \begin{bmatrix} x & x & 0 & 0 & 0 & 0 & 0 \\ 0 & x & x & 0 & 0 & 0 & 0 \\ 0 & 0 & x & x & 0 & 0 & 0 \\ 0 & 0 & 0 & x & x & 0 & 0 \\ 0 & 0 & 0 & 0 & x & x & 0 \\ 0 & 0 & 0 & 0 & 0 & x & x \end{bmatrix}$$

Solve $Ax = b$
for T^{n+1}

STABILITY CONDITION:

METHOD IS UNCONDITIONALLY STABLE
 Δt limited by accuracy, not stability.

Some ways linear systems arise in scientific computing

1. Discretization of differential equations

Finite difference methods yield narrow-banded matrices;
finite-element methods based on meshes produce sparse irregular ones



Some ways linear systems arise in scientific computing

2. Linearization of non-linear equations (Newton-Raphson)

Remember that for a scalar function the first-order Taylor approximation

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n)$$

gives way to Newton iterations

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

The corresponding multi-dimensional expansion yields

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J_F(\mathbf{x}_n)^{-1} F(\mathbf{x}_n)$$

which is equivalent to solving with respect to $\mathbf{x}_{n+1} - \mathbf{x}_n$ the linear system

$$J_F(\mathbf{x}_n)(\mathbf{x}_{n+1} - \mathbf{x}_n) = -F(\mathbf{x}_n).$$

Some ways linear systems arise in scientific computing

3. Least squares problems

Trying to fit parameters to noisy measurements via the least squares criterion

$$\min_x \|Ax - b\|_2^2$$

we apply the derivative condition:

$$\left((Ax - b)^T (Ax - b) \right)'_x = \left(A^T Ax^2 - (x^T A^T b + b^T Ax) + b^T b \right)'_x = 2A^T Ax - 2A^T b = 0 \Rightarrow A^T Ax = A^T b.$$

The latter condition is known as *normal equations*.

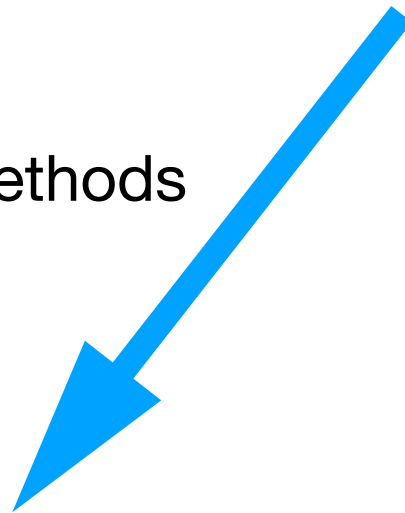
How to solve this ?

$$Ax = b$$

How to solve this ?

$$Ax = b$$

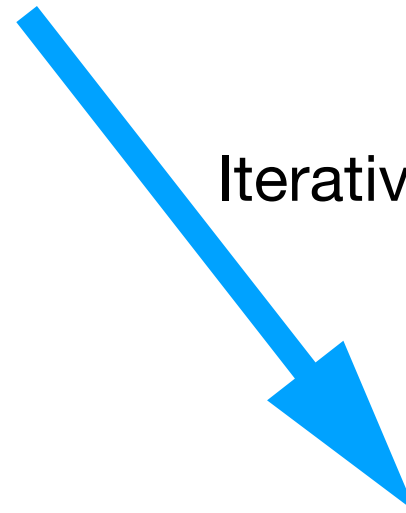
Direct methods



$$x = A^{-1}b$$

Exact solution,
except for rounding errors,
fixed number of computational steps

Iterative methods



$$x_t = f(x_{t-1}, A, b)$$

Often work better in practice,
due to care about numerical stability,
iteratively improve solution estimate

Direct methods

If all linear equations are uncoupled:

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

Solution is obvious

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

Solution is obvious

$$x_i = \frac{b_i}{a_{ii}}$$

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

Solution is obvious

$$x_i = \frac{b_i}{a_{ii}}$$

Can we diagonalize every matrix ?

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

Solution is obvious

$$x_i = \frac{b_i}{a_{ii}}$$

Can we diagonalize every matrix ?

$$P^{-1}AP$$

If all linear equations are uncoupled:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

Solution is obvious

$$x_i = \frac{b_i}{a_{ii}}$$

Can we diagonalize every matrix ?

Keyword: Jordan Normal Form

$$P^{-1}AP$$

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Upper triangular matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \ddots & \vdots \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

$$a_{ij} = 0, \forall i > j$$

$$a_{nn}x_n = b_n, \implies x_n = \frac{b_n}{a_{nn}}$$

$$a_{n-1,n-1}x_{n-1,n-1} = b_{n-1} - a_{n-1,n}x_n, \implies x_{n-1} = \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}}$$

Upper triangular matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \ddots & \vdots \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

$$a_{ij} = 0, \forall i > j$$

$$a_{nn}x_n = b_n, \implies x_n = \frac{b_n}{a_{nn}}$$

$$a_{n-1,n-1}x_{n-1,n-1} = b_{n-1} - a_{n-1,n}x_n, \implies x_{n-1} = \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}}$$

Quiz: What is the final complexity ?

Upper triangular matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \ddots & \vdots \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

$$a_{ij} = 0, \forall i > j$$

$$a_{nn}x_n = b_n, \implies x_n = \frac{b_n}{a_{nn}}$$

$$a_{n-1,n-1}x_{n-1,n-1} = b_{n-1} - a_{n-1,n}x_n, \implies x_{n-1} = \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}}$$

Quiz: What is the final complexity ?

Answer: $O(N^2)$

Gaussian Elimination

Exact solution,
except for rounding errors

$$A = LU$$

$$Ly = b$$

$$Ux = y$$

$$PA = LU$$

Not unique

Problems when A is singular

Solution is LUP - additional permutation matrix which aims to maximize absolute values of diagonal elements

Finding LU (LUP) has complexity $O(N^3)$

Special case: SPD matrices

Symmetric positive definite: if $A = A^T$ and $x^T A x > 0, \forall x \neq 0$

$$A = LL^*$$

Two times less memory than LU

Conditioning

Relative error: $\frac{||x - \hat{x}||}{||x||}$

$$r = b - A\hat{x} = Ax - A\hat{x} = A(x - \hat{x})$$

$$x - \hat{x} = A^{-1}r \implies ||x - \hat{x}|| = ||A^{-1}r|| \leq ||A^{-1}|| ||r||$$

$$\frac{||x - \hat{x}||}{||x||} \leq ||A^{-1}|| ||r|| \frac{||A||}{||b||}$$

Note:

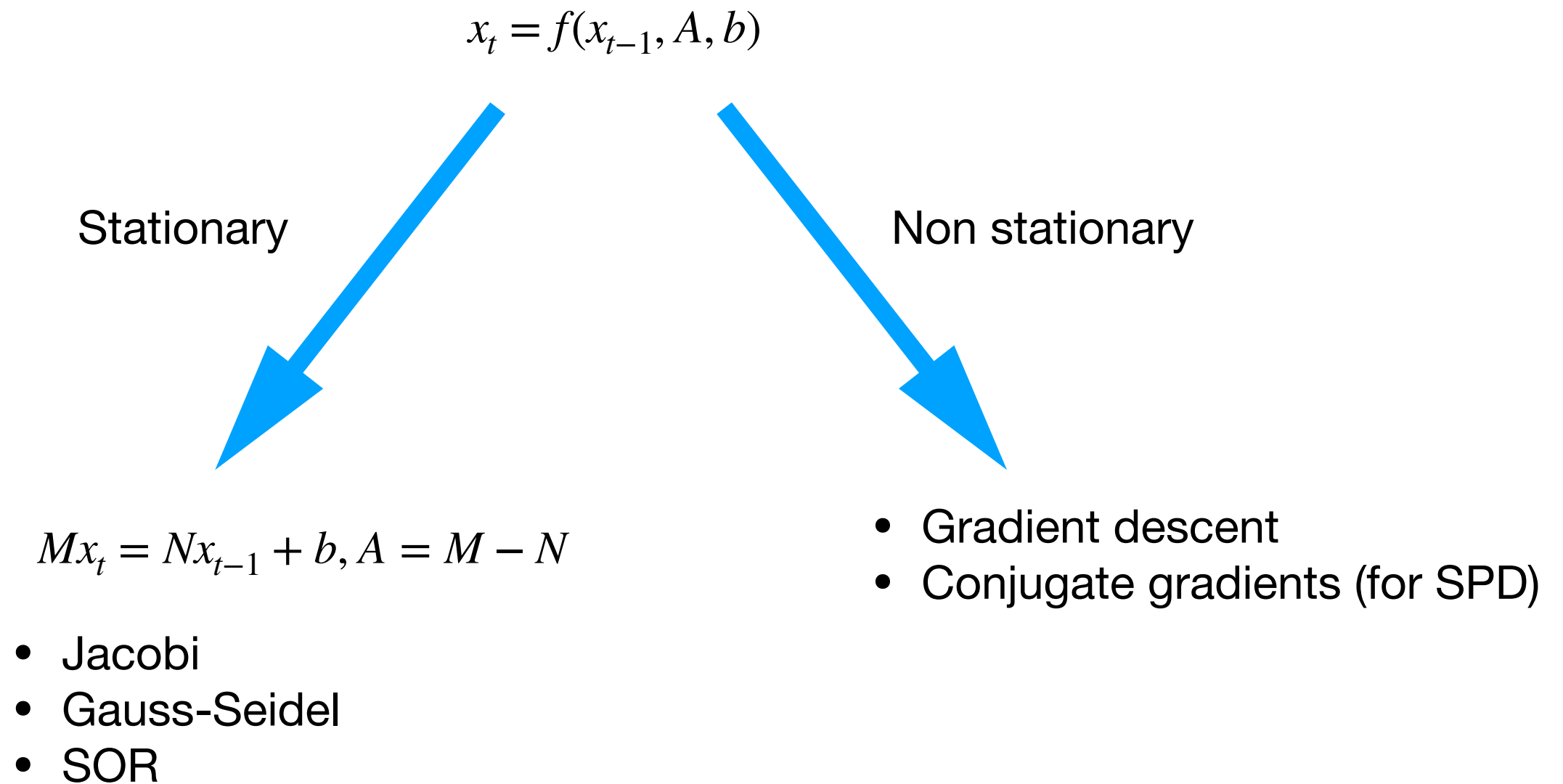
$$A\hat{x} = b + \Delta b \implies ||\Delta b|| = ||r||$$

Condition number: $k(A) = ||A|| ||A^{-1}||$

$$\frac{||x - \hat{x}||}{||x||} \leq k(A) \frac{||\Delta b||}{||b||}$$

the relative error in the solution is bounded by the condition number of the matrix A times the relative error in the residual.

Iterative methods



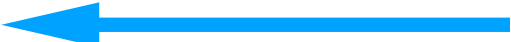
Stationary methods

$$Mx_t = Nx_{t-1} + b, A = M - N$$

We need initial guess for the solution: x_0

Error is: $e_0 = x - x_0$ 

We do not know the exact solution
(if we know, why do we care ?) and
cannot compute

$r_0 = Ae_0 = b - Ax_0$  We can compute

$$x = x_0 + e_0 = x_0 + A^{-1}r_0$$

Set $M\tilde{e} = r_k$

Preconditioner: M

Then $x_t = x_{t-1} + \tilde{e}$

$$A = D + E + F$$

Jacobi: $M = D$

Gauss-Seidel: $M = D + E$

SOR: $M = D + \frac{1}{\omega}E$

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ a_{21} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & 0 \end{bmatrix} E$$

$$+ \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ 0 & 0 & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix} F$$

$$+ \begin{bmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{bmatrix} D$$

Nonstationary methods

$$x_t = x_{t-1} + \alpha_t p_t$$

Gradient descent

$$x_t = x_{t-1} + \alpha_t p_t$$

$$p_k = r_k$$

or

$$M \tilde{e} = r_t$$

$$M = \alpha_t I$$

Gradient descent

$$x_t = x_{t-1} + \alpha_t p_t$$

$$p_k = r_k$$

or

$$M \tilde{e} = r_t$$

$$M = \alpha_t I$$

Conjugate gradient (SPD only)

$$x_t = x_{t-1} + \alpha_t p_t$$

$$p_t = r_t - \sum_{i < t} \frac{p_i^T A r_t}{p_i^T A p_i} p_i$$

$$\alpha_t = \frac{p_t^T r_t}{p_t^T A p_t}$$